

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

3. React JS

16. 專業ReactJS應用II

數字經濟核心科技系列：前端網站開發人員課程

(二) 進階網絡程式設計

3. React JS

前端開發與後端應用程式介面(API) 的結合方式

16. 專業ReactJS應用II

連接前端/後端應用程式的API

- **API(Application Programming Interface)** - 代表應用編程接口，是一套用於建立和整合應用軟件的定義和協議
- **API**讓產品或服務與其他產品和服務溝通，而不必知道對方是如何實現的
- 這可以簡化應用程式開發，節省時間和金錢
- 當設計新的(或管理現有的)工具和產品時，**API**能帶來靈活性；簡化設計、管理和使用；並提供創新機會



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

連接前端/後端應用程式的API

- 應用程式介面 (**API**) 能夠在維持安全與控制的同時，開放資源的存取權限，可以自行決定如何開放存取以及開放給誰
- **API** 安全的核心在於良好的 **API** 管理，這包括使用 **API** 閘道器
- 連接 **API** 並建立消耗 **API** 所暴露數據或功能的應用程式，可以通過分散式整合平台來實現，該平台將前端和後端連接在一起



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

連接前端/後端應用程式的API

- **RESTful API** 以及一般的網路服務，是一種將後端與前端開發者抽象分離的方式
- 讓前端開發者能夠專注於介面的設計，而無需編寫任何伺服器端的邏輯程式碼



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

前端和後端如何運作

前端和後端共同協作以實現使用者能夠存取關於前端和後端互動的詳細說明如下：

- 用戶端瀏覽器指向目標網站的網址(URL)
- 這使得瀏覽器向目標網站的伺服器發出一個或多個請求
- 瀏覽器等待收到伺服器的回應
- 每一個回應都被用來渲染網站的一個部分
- 用戶看到的是一個渲染後的網站頁面
- 用戶與頁面進行互動導致更多的請求被發送出去，以獲得更多的數據和顯示新的信息
- 前端和後端通過引起請求的發送和回應來實現這一點



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

前端和後端如何運作

- 用戶輸入一個URL，使瀏覽器發出請求到該網址
- 瀏覽器讀取傳入的HTML並在載入網站時，會注意到需要載入的資源，例如JS檔案、圖片或CSS檔案
- 瀏覽器會繼續為每個資源發出一個新的HTTP請求
- 這些請求未必傳送到同一個後端
- 可以從其他網站載入JS
- 也會使用快速且方便CDNs的來實現這些請求



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

前端和後端如何運作

- 用戶點擊一個網頁的連結，瀏覽器需要切換到新頁面，並請求相應的網址，相關網頁就會被加載並呈現出來
- JavaScript在網站上執行請求，在後台加載部分數據並回應，但不會重新加載整個頁面，這就是AJAX
- JavaScript也可以由用戶在網頁點擊某些內容觸發，而瀏覽器不需要切換到新頁面



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

前端和後端通信

在HTTP請求/回應的負載中

前端發送的:

- 不帶主體的簡單HTTP請求
- 表單數據
- JSON格式的數據

後端通常回應以下HTTP主體內容：

- HTML格式的回應
- 其他靜態檔案 (CSS、JS、圖片等)
- JSON格式的數據
- 完全無主體，僅狀態碼與標頭欄位



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

REST API - 連接前端/後端

- REST API(又稱 RESTful API)是一種應用程式介面 (API 或 Web API)
- 遵循REST架構風格的約束，並允許與RESTful網路服務進行互動
- REST全稱為「表述性狀態傳輸」(Representational State Transfer)
- 當客戶端發出請求時，REST API 會將資源狀態的表述傳輸給請求者或接收端



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

REST API - 連接前端/後端

資訊或其呈現方式會透過HTTP以下多種格式之一傳遞:

- JSON (Javascript Object Notation)
- HTML
- XLT
- Python
- PHP
- 純文本



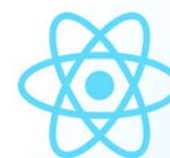
JSON 是最常用的檔案格式，因為它與程式語言無關，且人類和機器皆可讀取

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

設計良好的REST API的基本原則

- 在設計**API**時，必須注意：一個設計拙劣、缺乏文件或支援不足的**API**，除非被迫使用，否則沒有人會願意採用
- **API**設計必須兼顧功能性、易用性，並對開發者友善



ReactJS

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

設計良好的REST API的基本原則

- 良好的**API**能支援不同類型的客戶端，並能被其他開發者輕鬆理解
- 一個設計良好的**API**應該有以下元素:

1. 說明文件
2. 支持不同的數據格式
3. 統一資源標識符
4. 端點
5. 版本管理
6. 安全和認證
7. 可擴展性和靈活性
8. 錯誤處理和監控
9. HTTP狀態程式碼
10. 測試、穩定性和支持
11. 可維護性
12. 緩存
 - i. 過期
 - ii. 緩存控制
 - iii. 編碼
- IV. 最後修改時間
13. 搜索、過濾和排序
14. 分頁

16. 專業ReactJS應用II

REST API的文件

- **API說明文件**是一份技術手冊，其中包含如何有效使用和整合 **API** 的說明
- 這是一份全面的技術交付成果，詳細記錄了第三方開發者如何充分利用 **API**
- 該文件可能以程式碼示例、技術寫作或案例的形式呈現，詳細說明如何更好地使用**API**

16. 專業ReactJS應用II

REST API的文件

- API文件的最終目標是為第三方開發者(即API的主要使用者)提供最佳的體驗
- API使用者透過新增程式碼功能或建立新程式來解決編程挑戰，而不用詳細解讀API的隱藏資訊
- 開發者體驗(DX)至關重要，因為使用者期望API高效且易於使用
- 提供卓越開發體驗給使用者的重要一環，就是提供最新且準確的API文件

REST API支援的數據格式

REST API是連接客戶和伺服器的橋樑，所以**API**應該以一種對前端和後端都方便和可理解的格式發送和接收信息

這些是**API**中常用的數據格式:

1. 直接數據格式 - 如JSON、XML和YAML，這些是機器間的互動中常用的數據格式
2. 訂閱(Feed)數據格式 - 如RSS、Atom和SUP，用於博客、視頻分享和社交媒體，在前端界面與用戶互動，告知所有更新或變化
3. 數據庫數據格式 - 如CSV和SQL，用於數據庫和用戶之間的互動

REST API的統一資源標識符(URI)

- REST的關鍵原則是將API劃分為邏輯資源
- 統一資源標識符(Uniform Resources Identifier) , 即URI , 是一串識別資源的符號 , 用來允許開發者訪問該資源的表示方式
- 目前URI的結構和語法由RFC 3989標準定義 (<https://datatracker.ietf.org/doc/html/rfc3986>)

REST API的統一資源標識符(URI)

URI和URL(統一資源定位器)之間的區別

- URL確定了某個資源的位置，以及如何檢索它
例如: *http://example.com/example.html*
- URI通過其名稱來識別資源，但不一定顯示其位置。通常以“URN:”前綴開始
例如: *Urn:uuid:6e7bc280-7c3a-11d9-9889-08000200c9a66*

16. 專業ReactJS應用II

REST API的端點

- 端點指定了資源的位置以及第三方軟件對其的訪問方式，這些資源是通過URI存取的，HTTP請求被發送到該URI，並期望從該URI獲得回應
- REST API是應用程式接口，使用HTTP請求來GET、PUT、POST和DELETE數據

REST API的四個主要功能:

1. GET - 使用指定的URI從伺服器取得資源
2. POST - 向伺服器發送資源
3. PUT - 更新現有信息並上傳內容
4. DELETE - 刪除所有現有資源

GET	/photo	Lists all the photos in the database
DELETE	/photos/{photoid}	Deletes a photo based on their id
POST	/pic	Creates a photo
PUT	/photos/{photoid}	Method to update a photo
GET	/photos/{photoid}	Retrieves a photo based on their id

16. 專業ReactJS應用II

REST API的版本管理

- **API**常被更新，所以要進行版本管理並保留舊版本，以使用戶仍然可以依賴早期的版本，並在準備好時升級到任何新的版本
- 有4種常見的方法來對**API**進行版本管理:

1. **URI** - 版本會被明確地放在**API**的**URL**中，例如：

Api.example.com/v1/resources or example.com/api/v1/resources

2. 自定義標頭 - **API**也可以通過提供自定義頭文件，將版本號作為一個屬性包括在內來進行版本管理，例如：

curl -H "API-version:1.0" http://www.example.com/api/products

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

REST API的版本管理

3. 媒體類型版本(也被稱為內容否定或接受標頭) - 標題將修改為指定版本

例如：

```
curl -H "Accept:application/vnd.example.V2+json" http://www.example.com/api/products
```

4. 參數的版本化 - 把版本號作為一個查詢參數，例如：

```
http://www.example.com/api/products?version=1
```

16. 專業ReactJS應用II

REST API的安全和認證

- 必須時刻保護、保障和驗證誰在訪問網站以下是一些在開發API過程中確保網站安全的重要規則:
 1. 使用HTTPS - 安全的REST API應該提供HTTPS端點。一些認證憑證的方法，如API密鑰、密碼和JSON網絡令牌(JWT)
 2. 在HTTP請求中添加一個時間戳 - 與其他參數一起，為請求添加一個時間戳這樣，伺服器可以將請求的時間戳與當前的時間戳進行比較，並僅在特定時間範圍內(如1或2分鐘內)接受請求。這就消除了駭客試圖暴力破解系統的重放攻擊

16. 專業ReactJS應用II

REST API的安全和認證

3. 限制HTTP方法 - 建立允許的方法列表，並拒絕所有不在列表中的請求同時，確保調用者被授權在資源上使用傳入的HTTP方法
4. 設置輸入驗證 - 設置強大的驗證檢查，並在每一步都要驗證請求參數(輸入類型、長度、格式和範圍)。如果驗證失敗，立即拒絕請求
5. 使用OAuth 2.0授權框架允許應用程式以有限度的方式取得 HTTP 服務的存取權，可以通過資源所有者和HTTP服務之間的批准互動協調，或者允許第三方應用程式直接獲得

16. 專業ReactJS應用II

REST API的安全和認證

6. 收起敏感數據 - 用戶名、密碼、JWT和API密鑰不應出現在URL中
7. 執行安全檢查 - 建立安全檢查表，以減少SQL和XSS注入的可能性: 保護標頭，會話，cookie，允許參數; 在日誌中隱藏敏感數據; 使用CSRF令牌*; 驗證編輯和轉發; 實施強安全密碼檢查; 保護數據庫等。可以使用Brakeman**等自動化工具來完成

*跨站請求偽造 (亦稱為 CSRF) 是一種網路安全漏洞，允許攻擊者誘導使用者執行他們無意執行的操作。此漏洞使攻擊者能夠部分繞過同源政策，而該政策原本旨在防止不同網站之間互相干擾

** <https://brakemanpro.com/>

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

REST API的監控和錯誤處理

- 對訪問和使用趨勢的實時監控是必須的。必須對異常處理進行控制，這樣才能提供最新的信息，讓API客戶知道伺服器端發生了什麼
- 所有的異常都應該被映射到一個錯誤的有效載荷中，以確定錯誤來自哪裡，並為如何修復失敗的調用提供指引
- 可以使用Python構建API，比如Django
- 可以快速設置最常用的通知渠道，比如，電子郵件、Slack和Webhooks

16. 專業ReactJS應用II

REST API的監控和錯誤處理

- 如果需要查看統計數據，可以使用第三方服務，如Honeybager、Sentry和NewRelic

<https://docs.honeybadger.io/integrations/>

<https://docs.sentry.io/product/integrations/integration-platform/webhooks/>

<https://newrelic.com/>

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

REST API的HTTP狀態程式碼

- HTTP狀態程式碼，有時也被稱為互聯網或瀏覽器錯誤程式碼
- 由網絡伺服器發送的標準回應程式碼，用於識別所執行的操作是否成功，或者為什麼頁面或資源不能正常加載
- 可以幫助開發者進行故障排除和錯誤處理

16. 專業ReactJS應用II

REST API的HTTP狀態程式碼

- 回應分為四大類，並有超過50種回應程式碼
 - 下面是4個最常見的回應程式碼:
1. **2XX(成功)** - 200:OK。這是最常用的回應，用來通知請求已經成功了
 2. **3XX (重定向)** - 304:未修改。客戶端已經請求了緩存的數據，不需要再次傳輸
 3. **4XX (客戶端錯誤)** - 404:未找到。原始伺服器找不到目標資源的當前表示方式
 4. **5XX (伺服器錯誤)** - 503:服務不可用。由於臨時超載或計劃維護，伺服器目前無法處理該請求

16. 專業ReactJS應用II

REST API的維護和支援

- 維護和支援是成功部署後最重要的因素
- 這包括保持系統的運作，調查故障，修復錯誤，以及增加新的功能
- 保持系統運作的良好做法包括：
 1. 應用補丁和修復程序
 2. 平移能力
 3. 加強配置管理
 4. 保存有關該系統的知識庫

16. 專業ReactJS應用II

REST API的維護和支援

- 隨著用戶不斷提出新功能需求，系統必須能夠輕鬆適應變更
- 在設計系統時，務必保持簡單明瞭
- 簡潔且易於理解的系統更容易修改與維護

16. 專業ReactJS應用II

REST API的緩存

- 在快速訪問的存儲器中(內存中或內存外)緩存最頻繁或最近的請求數據，以快速滿足請求是很重要的
- 使用HTTP頭，原生伺服器表明回應是否可以被緩存，如果可以，由誰來緩存以及緩存多長時間

16. 專業ReactJS應用II

REST API的緩存

- 通常情況下，瀏覽器將所有的**GET**請求回應視為可緩存的
- **POST**請求回應在默認情況下是不可緩存的
- 如果讓回應成為可緩存的，可以在回應中添加一個**Expires**頭或**Cache-Control**頭
- 對**PUT**和**DELETE**請求的回應是不可緩存的

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

REST API的緩存

可以使用以下4個主要的HTTP回應標頭來控制緩存行為：

1. **Expires** - Expires HTTP標頭指定緩存的過期時間。過了這個時間，緩存的內容就會被認為是過時的，必須與原伺服器重新驗證

REST API的緩存

2. **Cache-Control** - 標頭的值由一個或多個逗號分隔的指令組成。這些指令決定了回應是否可以緩衝，如果可以，由誰來緩衝以及緩衝多長時間。可用的指令有：
- i. **public** - 表示資源可以被任何組件緩存
 - ii. **private** - 表示資源只可由客戶端和伺服器緩存
 - iii. **no-cache/no-store** - 表示資源是不可緩存的
 - iv. **max-age** - 表示緩存數據的最長的有效時間(秒數)。當時間過期時，客戶必須再次提出請求
 - v. **s-max-age** - 取代max-age共享緩存的最長有效時間，比如說在代理伺服器上，通常比客戶的本地緩存有更多的控制權，在這裡添加更長的值
 - vi. **must-revalidate** - 告訴伺服器，如果s-max-age/max-age已過，要重新驗證該資源

All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

REST API的緩存

- 3. Etag** - Etag值是隱藏的字串令牌，伺服器將其與資源關聯，以唯一識別該資源在其生命週期中的狀態。當資源發生變化時，Etag也會相應地改變
- 4. Last-Modified** - 回應的Date頭表示回應產生的時間，Last-Modified頭表示相關資源最後被修改的時間。最後修改時間不能晚於回應產生的時間

16. 專業ReactJS應用II

REST API的搜索、過濾和排序

- 對於任何搜索，所需的過濾條件、排序要求及進階搜尋功能，皆可作為搜索參數加在基礎URL上，協助簡化開發工作
- 對於過濾數據集，可以通過搜索語言為所有字段傳遞唯一的搜索參數
- URL參數是向REST APIs添加基本過濾功能的最簡單方法
例如: GET /dogs?breed=Akita&age<=2

16. 專業ReactJS應用II

REST API的搜索、過濾和排序

- 對於排序項目，可以使用排序URL參數
- 為了實現複雜的排序要求，將排序參數設置為一個逗號參數字段的列表，也可以加入一個負數來實現降序排序

例如: GET /dogs?sort=-breed,+name 或 GET /dogs?sort=breed_desc,name_asc

16. 專業ReactJS應用II

REST API的搜索、過濾和排序

- 除了基本的過濾器，還需要全文搜索的支援
- 如果需要在終端上進行搜索，可直接用搜索參數添加對過濾器和範圍的支援
- 也可以使用ElasticSearch或其他基於Lucene的搜索技術

<https://lucene.apache.org/>

<https://www.elastic.co/>



All Rights Reserved. Reprinting will not be tolerated.
版權所有，翻印必究

16. 專業ReactJS應用II

REST API的分頁

- 為了讓伺服器運作有效率，**API**需要將請求回應的項目分頁
- 大型的數據集，則需要將其分成小塊
- 為了提高性能並使其回應更容易處理，通過支持每頁參數使分頁更加靈活
例如：GET /dogs?page=3&per_page=20

16. 專業ReactJS應用II

參考

<https://technostacks.com/blog/best-frontend-frameworks/>

<https://www.sam-solutions.com/blog/best-frontend-framework/>

<https://www.simform.com/blog/best-frontend-frameworks/>

<https://dev.to/ericchapman/your-first-steps-with-express-js-34op>

[https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side JavaScript frameworks/Vue getting started](https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/Vue_getting_started)